

- [13] BLACK P E, Hyperedge; Hypergraph (Dictionary of Algorithms and Data Structures, NIST), <https://xlinux.nist.gov/dads/HTML/hyperedge.html>, 访问时间: 2026 年 09 月 26 日。
- [14] ÇATALYÜREK Ü V, AYKANAT C, Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication, IEEE Transactions on Parallel and Distributed Systems, 10(7): 673–693, 1999. DOI: 10.1109/71.780863.
- [15] VELDT N, BENSON A R, KLEINBERG J, Hypergraph Cuts with General Splitting Functions, SIAM Review, 64(3): 650–685, 2022. DOI: 10.1137/20M1321048.
- [16] DINIC E A, An algorithm for the solution of the max-flow problem with the polynomial estimation, Soviet Mathematics Doklady, 11: 1277–1280, 1970.
- [17] EYRAUD-DUBOIS L, MARCHAL L, SINNEN O, VIVIEN F, Parallel scheduling of task trees with limited memory, Inria Research Report RR-8606, 2014. <https://arxiv.org/abs/1410.0329>. DOI: 10.48550/arXiv.1410.0329.
- [18] SETHI R, ULLMAN J D, The Generation of Optimal Code for Arithmetic Expressions, Journal of the ACM, 17(4): 715–728, 1970. DOI: 10.1145/321607.321620.
- [19] JIN C, PUROHIT M, SVITKINA Z, VEE E, WANG J R, New Tools for Peak Memory Scheduling (arXiv:2312.13526, 2023), <https://arxiv.org/abs/2312.13526>, 访问时间: 2026 年 09 月 26 日。DOI: 10.48550/arXiv.2312.13526.
- [20] CURTSINGER C, BERGER E D, Coz: Finding Code that Counts with Causal Profiling, Proceedings of the 25th Symposium on Operating Systems Principles: 184–197, 2015. DOI: 10.1145/2815400.2815409.
- [21] ANGEL E, MORAIS S, REGNAULT D, A Bi-Criteria FPTAS for Scheduling with Memory Constraints on Graphs with Bounded Tree-Width, Euro-Par 2022: Parallel Processing. LNCS 13440: 136–151, 2022. DOI: 10.1007/978-3-031-12597-3_9.

附录 A 全量用例测试结果与数据说明

`data/all-results.csv` 保存三题共 1500 种图与核心数量组合的完整测试结果。每行记录包含题号、图编号、计算核心数、单核心基线 Makespan、多核 Makespan、额外 DDR 搬运字节数、端到端求解耗时及结果来源哈希；问题三另行记录关闭 L2 时的对照结果与 Cache 字节命中率。

由于全量 1500 行数据表格篇幅较大，完整数据清单独立排版于附录文档 `result-tables.pdf` 中供查阅，正文实验章节保留核心统计指标、均值、分位数与标准物理单位。数据表格中，执行时间以周期 (cycle) 为单位，数据量以字节 (B) 为单位，命中率以百分比呈现。正文所有聚合统计均基于未舍入的原始数据进行精确计算。数据来源文件及其 SHA-256 校验和在 `data/source-receipt.json` 中逐项注册，以确保实验记录的一致性与可溯源性。

附录 B 补充数学定理与推导证明

B.1 基于数据依赖的完成时间下界证明

本节假设每个需要讨论的原图中间张量至多由一个原始计算操作生成，不同原始张量标识不合并成同一逻辑数据标识。取出原始计算操作，只保留有依据的先后关系，包括原来就存在的计算关系，以及同一张量的生成操作必须先于读取操作的关系。不任意缩短经过原始 COPY 的路径。

在任何官方成功执行中，这些计算操作及其时长都被保留，每个核心的每条 Pipe 也仍同时至多处理一个操作。直接取该次执行中各计算的核心和开始、完成时刻，就得到满足保留关系的一组安排。因此，在这些必要关系下求得的最小必要时间，不会大于官方完成时间。

对于分配到不同核心的计算操作，必须核对两者之间的依赖怎样在官方执行中保留。问题一要求相关前驱 Task 完成后，后续 Task 才允许开始；问题二、三要求源 COPY_OUT 完成，再经过 500 cycles，目标 COPY_IN 才可参与就绪判断；实际开始仍需满足其余数据依赖、执行单元和存储条件。接收端若因容量不足而需要再次读入数据，还受官方 Step 2 次序和 MTE2 操作队列顺序限制，不能跳过所需的基础 COPY_IN。上述必要关系均从官方成功执行所满足的条件中提取。

B.2 时间区间工作量与不可分割性下界

在 B.1 的计算操作之间的数据依赖图上，为操作 v 定义两个不包括自身的时长： h_v 为它之前最长依赖路径的时长， t_v 为它之后最长依赖路径的时长。无相应前驱或后继时取 0。递推式为：

$$h_v = \max_{u \rightarrow v} (h_u + d_u), \quad t_v = \max_{v \rightarrow w} (d_w + t_w). \quad (\text{B.1})$$

本节 h_v 以 cycle 计，与第 6 章按字节定义的子树存储量 h_i 是不同的局部记号。取 $a, b \geq 0$ ，选定一种 Pipe p ，考虑非空操作集合：

$$S = \{v : p_v = p, h_v \geq a, t_v \geq b\}. \quad (\text{B.2})$$

任何成功执行中， S 的操作都只能完整安排在 $[a, M - b]$ 内：它们之前至少需要 a 个周期，之后至少需要 b 个周期。同类 Pipe 共有 K 个，每个操作又不能拆成多段交给不同核心。因此，除了总工作量，还可以利用操作不可分割这一条件。

将 S 中 n 个操作的时长降序排列为 $d_{(1)} \geq \dots \geq d_{(n)}$ 。定义：

$$A_K(S) = \max_{\substack{r \geq 1 \\ m = (r-1)K + 1 \leq n}} \sum_{j=m-r+1}^m d_{(j)}. \quad (\text{B.3})$$

于是：

$$M \geq a + b + \max \left\{ \left\lceil \frac{\sum_{v \in S} d_v}{K} \right\rceil, A_K(S) \right\}. \quad (\text{B.4})$$

证明. 令 $x = M - a - b$ ， K 个同类 Pipe 在该区间内最多提供 Kx 个计算周期，得到第一项。取前 $m = (r-1)K + 1$ 个操作，至少一个核心要执行其中 r 个；即使选其中最短的 r 个，也至少需要 $\sum_{j=m-r+1}^m d_{(j)}$ 周期。对允许的 r 取最大得到第二项。两项都限制同一个区间的长度，故取最大值，不相加。最后只加一次前后所需的 $a + b$ 。□

例如，两个核心执行三个不可拆分操作，时长分别为 10、8、7 cycles。总工作量给出 13 cycles 的下界，上式利用不可分割性给出 15 cycles。达到 15 仍可能受其他依赖和资源限制，这个例子只说明下界为何能加强。

图上递推 h_v, t_v 需要 $O(n + m)$ 时间；固定 a, b 后，检查集合成员需 $O(n)$ ，对选中操作排序需 $O(|S| \log |S|)$ 。枚举所有 a, b 还要另计成本。若 S 为空，不能仅凭 $a + b$ 得到有效下界。

B.3 计算工作量与跨核心等待约束

考虑一串按数据依赖拓扑排序、先后存在确定依赖关系的特定计算算子序列 $u_0 \rightarrow u_1 \rightarrow \dots \rightarrow u_m$ ，将其作为相邻时间区间的边界分隔操作。为保证后文各区间的完成时间下界能够直接线性相加，所选区间必须严格互不重叠且在拓扑上具有确定的前后依赖链关系。任取若干缺乏全局全序关系的汇合节点并不足以支撑区间的可加性。

对于由相邻分隔算子 u_{i-1} 与 u_i 划定的局部时间区间，考察落在该区间内部且绑定在同一种执行流水线（Pipe） p 上的非空计算操作子集 $S \subseteq V_{\text{comp}} \setminus \{u_{i-1}, u_i\}$ （不含分隔操作自身）。对每个操作 $v \in S$ ，严格定义其局部的内部前后依赖耗时：

- 前端内部计算耗时 $h_i(v)$ ：以左边界分隔操作 u_{i-1} 完成时刻为时间零点，定义为从 u_{i-1} 到 v 的内部前驱计算操作路径时长（仅累加内部前驱计算算子的固有执行时长，不含 u_{i-1} 与 v 自身；若为无中间计算操作的直接依赖则取 0；若存在多条路径则取该 v 对应的最长路径）；
- 后端内部计算耗时 $t_i(v)$ ：定义为从 v 完成到右边界分隔操作 u_i 开始时刻所需的内部后继计算路径时长（仅累加内部后继计算算子的固有执行时长，不含 v 与 u_i 自身；若为直接依赖则取 0；若存在多条路径则取该 v 对应的最长路径）。

不能将附录 B.2 中相对于全图源汇点全局定义的 h_v, t_v 直接代入局部区间。选取统一阈值对 $a, b \geq 0$ 时，必须取非负整数周期 ($a, b \in \mathbb{N}$) 以保证后续有效窗口 $z = x - a - b$ 为确定离散整数周期，且必须满足集合 S 内的所有操作 $v \in S$ 均有 $h_i(v) \geq a$ 且 $t_i(v) \geq b$ （等价于 $a \leq \min_{v \in S} h_i(v)$ 且 $b \leq \min_{v \in S} t_i(v)$ ）；亦可先选定非负整数阈值 $a, b \geq 0$ ，再据此筛选满足条件的非空同流水线集合 $S = \{v \in V_{\text{comp}} \setminus \{u_{i-1}, u_i\} : \text{pipe}(v) = p, h_i(v) \geq a, t_i(v) \geq b\}$ 。严禁将 a, b 定义为整个集合 S 的最长依赖路径。集合 S 中的每个受约束操作，必须存在经附录 B.1 官方硬件语义验证的边界依赖路径。若考察的区间处于计算图最前端（无前驱分隔操作），则令 $\ell = 0$ 且 $a = 0$ ；若处于计算图最末端（无后继分隔操作），则令 $r = 0$ 且 $b = 0$ ；当区间两端均存在确定分隔操作时，令 $\ell = 1, r = 1$ 。

对于被分配至不同核心的计算操作，跨核心硬件依赖等待常数记为 τ_q （依据附录 B.1 经官方验证的硬件语义：问题一跨 Task 硬件同步屏障为 $\tau_1 = 1000$ cycles；问题二与问题三跨核心搬运等待下界为 $\tau_2 = \tau_3 = 500$ cycles；该等待下界仅在硬件规则明确支持的范围内生效）。为精确区分操作分配核心与边界核心的关系，定义逐操作的核心归属指示变量：若左边界分隔操作 u_{i-1} 存在 ($\ell = 1$) 且操作 v 所在核心与左边界核心不同（即 $\text{core}(v) \neq \text{core}(u_{i-1})$ ），则令 $\delta_L(v) = 1$ ，否则令 $\delta_L(v) = 0$ ；类似地，若右边界分隔操作 u_i 存在 ($r = 1$) 且 $\text{core}(v) \neq \text{core}(u_i)$ ，则令 $\delta_R(v) = 1$ ，否则令 $\delta_R(v) = 0$ 。结合内部依赖阈值与跨核等待，当存在对应侧分隔操作时，任意 $v \in S$ 的开始执行时刻与完成时刻满足：

$$\text{start}(v) \geq \text{left_end} + \max(a, \delta_L(v)\tau_q), \quad \text{finish}(v) \leq \text{right_start} - \max(b, \delta_R(v)\tau_q), \quad (\text{B.5})$$

其中 left_end 为左边界完成时刻， right_start 为右边界开始时刻（若某一侧不存在分隔操作，则直接使用计算图整体执行的起止边界）。

指标 $\ell, r \in \{0, 1\}$ 仍用于表征边界存在性及定义跨核心残余等待。扣除区间内部计算已覆盖的重叠时间后，同时不同于两个存在边界核心的其他核心所受的残余纯等待时间下界为：

$$c(a, b) = \ell(\tau_q - a)_+ + r(\tau_q - b)_+, \quad (\text{B.6})$$

其中 $(x)_+ = \max(x, 0)$ 。当两端边界指派在同一核心时，该核心自身完全不受这两个跨核等待约束，拥有完整的有效计算窗口 z ；当两端边界位于不同核心时，各边界所在核心分别只受另一端的跨核约束，而其余 $K - 2$ 个既不承担左边界也不承担右边界的核心则同时承受两端的残余等待限制。

设该局部时间区间的物理跨度为整数周期 $x \geq a + b$ ，扣除前后内部计算耗时后，可供集合 S 中操作并发执行的有效时间窗口为整数周期 $z = x - a - b \geq 0$ 。对应流水线上的总计算工作量为 $W(S) = \sum_{v \in S} d_v$ 。若前后两个分隔操作指派在同一核心，该核心在窗口内最多可提供 z 个周期，其他核心受跨核传输等待约束，各至多提供 $(z - c)_+$ 个有效计算周期，故有：

$$W(S) \leq z + (K - 1)(z - c)_+. \quad (\text{B.7})$$

若前后分隔操作位于不同核心（此时必有 $K \geq 2$ ），令 $A = (\tau_q - a)_+$ ， $B = (\tau_q - b)_+$ 。各核心可供同类流水线使用的总周期数至多为：

$$(z - A)_+ + (z - B)_+ + (K - 2)(z - A - B)_+. \quad (\text{B.8})$$

由于实数运算中恒有 $(z - A)_+ + (z - B)_+ \leq z + (z - A - B)_+$ ，上述单核心主导不等式对跨核心情形同样构成有效上界约束。对于离散整数周期，满足工作量约束所需的最小有效窗口时长为：

$$\phi_{K,c}(W) = \min \left\{ W, \left\lceil \frac{W + (K - 1)c}{K} \right\rceil \right\}. \quad (\text{B.9})$$

结合单个算子在物理流水线上不可跨核心拆分的离散限制，该局部区间的最小物理耗时下界为：

$$x \geq a + b + \max\{\phi_{K,c(a,b)}(W(S)), A_K(S)\}. \quad (\text{B.10})$$

仅当各个区间之间严格互不重叠且由拓扑依赖链串联时，各区间的下界与分隔算子自身的执行周期方可直接线性求和累加（分隔算子自身执行时长另行计入）。若仅考察局部相互重叠的滑动窗口，则只能取各窗口下界的最大值，绝不可重复累加。此外，问题一中同一核心内部不同 Task 切换所引入的 100 cycles 切换开销，由于具体方案未必在所选算子边界处切分子图，不能作为必要下界无条件计入。若上述任一前提条件无法满足，则该下界公式仅在其实际成立的受限子图范围内生效，不予强行推广。

由上述推导可见，与前后计算重叠的等待时间已被严格扣除，各时间段仅计算一次。本节为离线补充理论推导，正文下界统计使用的是第 7 章所列已有版本的结果。

B.4 调度状态信息充分性的判定定理与证明

本节讨论一个尚未由本文主算法一般实现的理论问题：能否仅保存方案的部分抽象状态信息，即可完全无损地判定后续结构修改的可行性及其执行结果。两份方案当前离散模拟的 **Makespan** 相同，并不足以说明它们在未来的修改演化中等价。例如，两份方案可能具有不同的核心内子图执行拓扑次序，某项算子迁移修改在第一份方案中合法可行，而在第二份方案中却可能诱发循环依赖死锁。

为了形式化分析状态信息的充分性，必须同时固定输入计算图 G 、计算核心总数 K 、赛题问题编号 q 以及官方硬件配置 (**config**)，并固定官方评测源码版本 (**Commit SHA**)、底层解释器与执行环境，以及完整的确定性实现约定（包括稳定的算子排序、字典迭代顺序、平局规则以及浮点精度）。需要明确的是，源码提交哈希本身并不单独保证跨平台环境下的位级一致性，确定性行为的严格成立依赖于执行环境与实现约定的共同固定。设 $\mathcal{P}$ 为声明的合法调度方案域，定义一组允许对方案施加的确定性原子修改动作集合 $\mathcal{A}$ （例如在声明的结构范围内移动、合并或拆分子图；每项动作的参数含义与执行逻辑严格确定）。对于任意动作 $a \in \mathcal{A}$ ，记能够合法执行该动作的方案子集为 $D_a \subseteq \mathcal{P}$ 。我们假设动作转移具有确定性，且声明的方案域 $\mathcal{P}$ 对允许动作封闭，即：

$$\forall a \in \mathcal{A}, \forall P \in D_a \implies a(P) \in \mathcal{P}. \quad (\text{B.11})$$

设 $\Phi(P)$ 为从方案 P 中提取的观测结果。若仅考察正常完成的执行，可取观测结果为 **Makespan**；若涵盖可能出现的执行异常，则须将成功或失败的离散状态编码入观测向量中，而不能为未定义的执行赋予臆想的周期数值。

令 $\mathcal{L}(P)$ 为从方案 P 出发、能够依次连续施加的所有有限动作序列集合（包含不施加任何修改的空序列 ϵ ）。两份方案 P 与 Q 在动作系统 $\mathcal{A}$ 下等价，要求它们诱导的可执行动作序列集合完全相同，且在任意相同动作序列作用后的观测结果完全一致：

$$\mathcal{L}(P) = \mathcal{L}(Q), \quad \Phi(w(P)) = \Phi(w(Q)), \quad \forall w \in \mathcal{L}(P), \quad (\text{B.12})$$

其中 $w(P)$ 表示按序列 w 中的动作顺次对方案 P 实施转移。

命题 B.1（状态充分性条件）。在同时固定输入计算图 G 、计算核心数 K 、问题场景 q 、官方硬件配置参数、评测源码版本、参考执行环境以及确定性约定的条件下，假设动作转移具有确定性且方案域 $\mathcal{P}$ 对 $\mathcal{A}$ 封闭。设 $S: \mathcal{P} \rightarrow \mathcal{X}$ 为对方案提取的抽象状态函数。若针对每一个原子动作 $a \in \mathcal{A}$ ，均存在前置可行性判定函数 $d_a: \mathcal{X} \rightarrow \{\text{True}, \text{False}\}$ 、状态更新函数 $\bar{a}: \mathcal{X} \rightarrow \mathcal{X}$ 以及全局观测恢复函数 $o: \mathcal{X} \rightarrow \mathcal{Y}$ ，使得对于方案域内的任意方案 $P \in \mathcal{P}$ 均满足：

$$\begin{aligned} P \in D_a &\iff d_a(S(P)), \\ S(a(P)) &= \bar{a}(S(P)) \quad (\forall P \in D_a), \\ \Phi(P) &= o(S(P)). \end{aligned} \quad (\text{B.13})$$

则 $S(P) = S(Q)$ 必能充分推导出方案 P 与 Q 的行为等价性（即 $\mathcal{L}(P) = \mathcal{L}(Q)$ 且 $\Phi(w(P)) = \Phi(w(Q))$ 对任意 $w \in \mathcal{L}(P)$ 恒成立）。

证明. 当动作序列为空序列 ϵ 时，由 $S(P) = S(Q)$ 及观测恢复条件直接得到 $\Phi(P) = o(S(P)) = o(S(Q)) = \Phi(Q)$ 。假设对于任意长度不超过 k 的可执行动作序列 $w = (a_1, a_2, \dots, a_k)$ ，均有 $w \in \mathcal{L}(P) \iff w \in \mathcal{L}(Q)$ ，且转移后方案仍满足 $S(w(P)) = S(w(Q))$ 。对于任意追加动作 $a_{k+1} \in \mathcal{A}$ ，由于可行性判定函数 $d_{a_{k+1}}$ 仅依赖于状态 S ，由 $S(w(P)) = S(w(Q))$ 可知：

$$w(P) \in D_{a_{k+1}} \iff d_{a_{k+1}}(S(w(P))) \iff d_{a_{k+1}}(S(w(Q))) \iff w(Q) \in D_{a_{k+1}}. \quad (\text{B.14})$$

因此长度为 $k+1$ 的序列在两方案上同时可行或同时不可行。若该动作可行，由于方案域封闭性，转移后方案依然落在 $\mathcal{P}$ 内，由状态更新等式可得：

$$S(a_{k+1}(w(P))) = \bar{a}_{k+1}(S(w(P))) = \bar{a}_{k+1}(S(w(Q))) = S(a_{k+1}(w(Q))). \quad (\text{B.15})$$

进而通过观测恢复函数导出两者的观测完全一致：

$$\begin{aligned} o(S(a_{k+1}(w(P)))) &= o(S(a_{k+1}(w(Q)))) \\ &= \Phi(a_{k+1}(w(P))) \\ &= \Phi(a_{k+1}(w(Q))). \end{aligned} \quad (\text{B.16})$$

由数学归纳法可知，等价性结论对任意有限长度的可执行动作序列均严格成立。 $\square$

需要明确的是，命题 B.1 仅给出了状态信息足以支撑无损预测的充分条件，并未证明该状态表示在信息论或状态复杂度意义下达到了最小表示（极小商空间），亦未假定微架

构全局响应具有简单的可加性。若要证明状态的最小性，必须进一步证明任意两个不同的抽象状态均存在某条测试动作序列能够将其观测区分开。本文求解器在实现上并未构建此类通用行为商空间，而是针对具体的搜索算子实施轻量级局部校验与字节级哈希去重；本节推导系基于本文模型独立核验的条件性理论界限，旨在为未来探索状态折叠与受控搜索空间压缩提供严谨的判定基准。

附录 C 核心算法实现与实验复现说明

C.1 程序源码与正文算法对应关系

各问题的算法主实现、评测上限如表 C.1 所示，正文伪代码与源码的详细映射关系如表 C.2 所示。

表 C.1 问题一至问题三的稳定实现版本与复现入口

问题	固定源码提交	模块入口	在线评价次数上限
P1	834d8c957538ee06 9c66aadac9509552 a4cc69d7	src.q1.branch_re fine	10 次 E1
P2	c66559a6f8a31ef7 b4720e1f7c3c28d6 1f8dff3f	src.q2_nikolasta rx.adaptive_hype rgap_guarded	3 次 E2
P3	311322b996c0948e 8a6a9c7ec6ddfe6a e41fbee1	src.q3.forest_so lve	3 次 E0

表 C.2 正文各算法伪代码与程序源码对应映射关系。

算法	算法名称	源码对应位置	来源提交
算法 4.1	P1 统一细化流程	src/q1/branch_re fine.py::solve	P1, 见上一表
算法 4.2	局部支路重分配候选构造	src/q1/branch_a i d.py::construct (359–438 行)	P1, 见上一表
算法 5.1	二核心负载约束超图割	src/q2_nikolasta rx/binary_hyperc ut.py::load_guar ded_cut	P2, 见上一表
算法 5.2	自适应保护超图分核	src/q2_nikolasta rx/adaptive_hype rgap_guarded.py: :build	P2, 见上一表

表 C.2 正文各算法伪代码与程序源码对应映射关系。(续)

算法	算法名称	源码对应位置	来源提交
算法 6.1	森林拓扑指标递推与遍历序列构造	src/q3/forest_memory_order.py::construct (153–192 行)	P3, 见上一表
算法 6.2	森林候选池受限在线评估	src/q3/forest_solve.py::evaluate_candidates	P3, 见上一表

正文各伪代码块系为阐释核心控制流与数学决策而提取的高层抽象，完整程序实现、模块接口与依赖环境均归档于程序附件包 `attachments/v10/source-code-v10.zip` (详见附录 E)，其与表 C.1、表 C.2 标定的固定历史提交严格对应。官方代码哈希为 `de11a83db8d7c47ed328b15a7df71d613a833b16cd23ee9fe877999578a1ace0`，官方硬件配置哈希为 `dcd10de54b23f8366428fb24e828812b1da9549e6eae4a3c3f38604fe5ae77b9`。已有实验结果可运行 `python3 paper/manuscript-v1/scripts/export_tables.py` 从冻结数据直接导出，不调用求解器或评价器。

C.2 问题二理论下界证书来源与校验

第 7 章中与 P2 全量方案 U 比较所采用的全局下界证书 $L(G, K)$ ，源自固定提交 `00d311ed0eea0fd86f9840df406956041a7c192a` 中的文档 `docs/a/q2-nikolastarx/OPTIMALITY_BOUNDS.md` 与生成脚本 `src/q2_nikolastarx/global_bounds.py`。该证书文件的 SHA-256 校验和为 `fe25f7b7737dbd3d841284bc5939242982613f64e0cffa9bd3d25df1f4a0c96b`，归档于数据资产 `results/a/q2-nikolastarx/goal-20260924/global-bounds.json`。

在固定复核脚本 `results/a/q2-nikolastarx/hypergap-full500-audit-20260925/audit.py` 中，该证书直接关联至新方案 U 的逐用例评估数据。该下界覆盖 100 张计算图在 1~5 核下的完整 500 个评测点，是针对任意合法无死锁多核划分可行域 $\mathcal{F}_2(G, K)$ 建立的全局理论下界；其与第 6 章针对固定完整多核候选方案 P 所建立的 FIFO 与依赖必要不等式下界 $L(P)$ 具有不同内涵，而附录 B.3 则属于未产生本次统计的新离线理论补充推导（不构成单核方案证书），三者概念严格分离，第 7 章的数值界限比较仅采用上述全局证书 $L(G, K)$ 。

附录 D 图件数据定义与微架构诊断示例

D.1 图件视觉符号与数据来源说明

正文图件中的数据来源与视觉符号定义如下：

- 计算图片段：**图 4.3-1 中以 `case_026` 为例展示了原图局部的 8 个 ADD 算子，图中完整保留原始节点数字编号与各张量的传输字节数；黑色实线箭头表示算子间的数据与控制依赖，橙色虚线表示跨子图通信数据流；边界引出的点线端口用于连接局部裁剪区域外的依赖算子（表明其在原图中具有前驱或后继，并非孤立节点）。
- 执行时间线与状态：**图 4.5-2、图 5.4-4 等展示的真实执行时间线严格绑定相同输入、方案、核心数量和官方配置，各时间区间均由官方微架构模拟器记录的真实底层事件直接导出。时间线中的空白区间仅代表对应流水线在当前周期未记录到活跃操作占位，并不

等价于计算核心整体休眠或 Task 停止，只有在存在仿真事件证据时才对等待原因进行解释。

3. **理论与教学示例**：第 5 章中的手算张量生命周期图与第 6 章中的树状示例明确注明为教学与数学原理示例，横轴为离散处理步骤，纵轴为内部张量占用的理论字节数，用于辅助阐明状态递推与定序准则，不代表官方模拟器的实际执行时刻。
4. **Cache 行为标注**：Cache 相关图件中严格区分了发起请求时刻的查询动作与 DDR 传输完成后的写入插入时刻。

D.2 算例 case_051 微架构执行时间线对比

图 D.2-1 展示了针对算例 case_051（3 核）进行的一组微架构诊断性对比（对应图件来源提交 bc2a42c277b9d70c51f3fc6974275354df33ee26，数据来源提交 059056ce1ee999e634049e5e5baf72566832effe），呈现了在相同链-核映射下，不同子图分组方案（control 与 paced 候选）在彩色流水线操作与灰色 Task 执行区间两层结构中的微架构事件分布。

在图 D.2-1 中，两方案在前 12,000 周期的首轮执行阶段，三个关键事件时刻严格相同：大输入搬入完成于 6,570 周期、首轮 Task 完成于 10,028 周期、远端部分结果返回完成于 11,040 周期。首轮之后两者的事件推进呈现不同，最终汇合所有部分结果的末条 Task 起始时刻由 291,076 周期提前至 278,472 周期（提前 12,604 周期），整图 Makespan 从 291,222 周期缩短至 278,618 周期（降幅 4.33%），同时额外 DDR 搬运量由 9,045,304 B 变为 9,045,350 B（增加 46 B）。两方案的完整求解器物理墙钟耗时分别为 0.546 s 与 0.618 s（略有增加，不作提速宣称）。

必须指出，上述对比系基于单一具体用例事件记录的诊断性观察，两方案具有相同链-核映射但子图分组不同，比较观察不代表仅改变了执行顺序；单用例局部事件提前不能泛化为全用例规律，亦不能直接作为判定硬件资源争用因果关系的依据。

附录 E 核心算法程序附件说明

本文核心算法程序完整代码已归档至独立附件包 attachments/v10/source-code-v10.zip。包内收录三题固定历史提交下的源算法模块、附录 C 标定的 7 个核心算法原始及带规范声明展示副本、官方评价器与依赖源码，以及完整的 SHA-256 哈希校验清单。详细文件目录与运行环境限制参见包内 README.md。

本附件定位为源码审阅附件。其中问题二入口执行所需的原 Git 对象、经过认证的原生二进制文件及最终生产运行环境验证，须按 README.md 说明由各程序责任人独立补齐。

附录 C 标定的 7 个核心算法模块在编程实现中使用了 OpenAI 的 GPT-6 Astra (gpt-6-astra, 2026-09-03) 提供辅助支持，展示副本头部已附规范声明；此事实严格限定于上述 7 个模块，不外推至其他队友编写的源码。

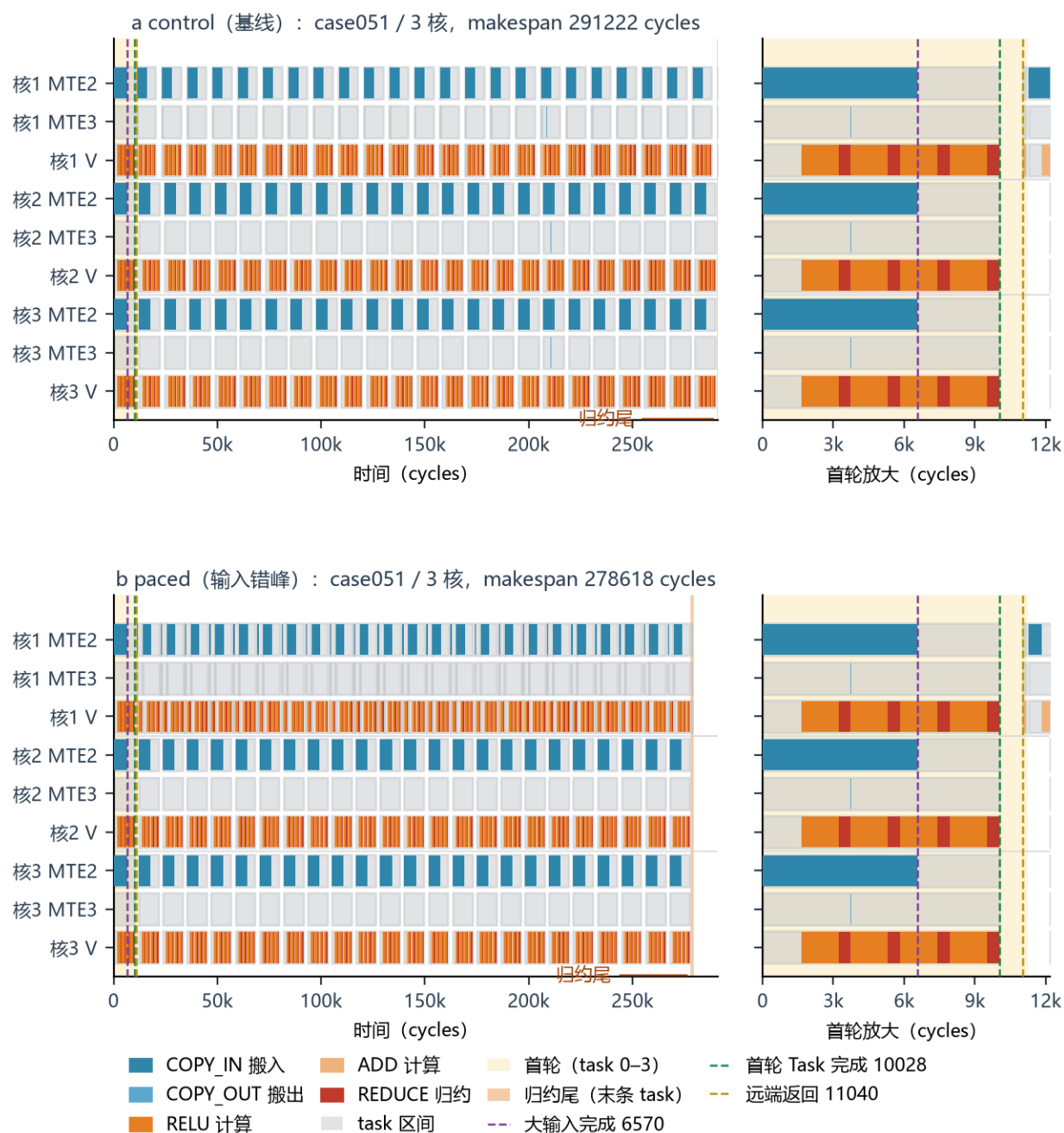


图 D.2-1 算例 case_051 (3 核) control 对照组 (291,222 cycles, 额外 DDR 搬运量 9,045,304 B) 与 paced 步调组 (278,618 cycles, 额外 DDR 搬运量 9,045,350 B) 的执行时间线对比。图内呈现彩色流水线操作与灰色 Task 执行区间两层结构；Core 1-3 对应官方 Core 0-2；“归约尾”标注最终汇合 Task 的起始时刻。